



Análise e Desenvolvimento de Sistemas
PROGRAMAÇÃO ORIENTADA A OBJETOS COM C#
RELATÓRIO DE AULAS PRÁTICAS

Nome: _____

RA: _____

Polo de matrícula: _____

Local da realização da Aula Prática: _____

Ano da postagem: _____

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): CONSTRUINDO UM PROGRAMA DE CONSOLE EM C# 14, COM .NET 10: TIPOS, NULLABILIDADE, PATTERN MATCHING, RECORDS E PROPRIEDADES

ORIENTAÇÕES:

Cada aluno deve produzir um relatório sucinto (2 a 3 páginas) contendo:

1) Um resumo teórico

No resumo teórico, apresente, com palavras próprias:

- O que é **Programação Orientada a Objetos (POO)** e como os conceitos de **classe, objeto, propriedade, método e encapsulamento** aparecem em um programa simples.
- O que são **tipos por valor** e **tipos por referência**, e como isso afeta null e o comportamento do código.
- O que é **nullabilidade** (por exemplo, string?) e o que são os operadores ?? e ??= (para lidar com valores nulos).
- O que é **pattern matching**, com foco em switch como **expressão** e no uso de padrões com when.
- O que são **records**, por que têm **igualdade estrutural** e como funciona a **with-expression** para criar cópias modificadas.
- O que é uma **propriedade com validação** (por exemplo, lançar ArgumentNullException ao receber null) e como isso se relaciona com encapsulamento.
- O que é a **atribuição condicional nula** (usar ?. no destino de uma atribuição) e em que situação ela evita if repetitivo.

Observação técnica para embasamento: o suporte do **C# 14** está associado ao **.NET 10 ou superior**.

2) O código-fonte comentado

- Inserir **o código-fonte completo** produzido na atividade (o arquivo principal do console).
- Comentar as partes centrais, explicando por que cada trecho existe e o que se espera que apareça na saída do console.

3) As referências (ABNT)

O aluno deve listar **os livros e as documentações** consultadas, com referências em **ABNT**.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	Clareza, correção conceitual e conexão entre POO, nullabilidade, records e propriedades.
A estrutura e a organização do código	3,0	Nomes adequados, organização legível, comentários úteis e ausência de “atalhos” confusos.
O funcionamento da solução	3,0	Compilação e execução sem erros, com todas as etapas implementadas e verificadas no console.
A criatividade e os aprimoramentos	2,0	Extensões coerentes, como ler valores do usuário, adicionar testes extras, ou incluir novos casos no switch.

Nota Final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão a nota reduzida proporcionalmente.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): DESENVOLVENDO UM PROGRAMA DE CONSOLE EM C#: AS CONDICIONAIS E OS LAÇOS (COM .NET 10 E C# 14)

ORIENTAÇÕES:

Cada aluno deve produzir um relatório sucinto (2 a 3 páginas) contendo:

1) Um resumo teórico

Explicar, com palavras próprias:

- O que são **as instruções condicionais** (if, else if, else) e como elas controlam o fluxo do programa.
- O que é o **switch** e em que situação ele costuma ser mais conveniente do que uma sequência de if/else.
- O que são **os laços de repetição** (for e while) e a diferença entre “repetir um número conhecido de vezes” e “repetir enquanto uma condição for verdadeira”.
- O que significa o código ser **case-sensitive** (por exemplo, variáveis com a minúsculo e com A maiúsculo serem nomes diferentes).
- O papel de **ponto e vírgula (;)** e de **chaves ({ e })** na estrutura do código.
- A relação desses recursos com a POO: como **métodos** e **classes** podem organizar um programa que inicialmente foi escrito no arquivo principal.

2) O código-fonte comentado

- Inserir **o código-fonte completo** produzido na atividade.
- Comentar as partes principais, explicando:
 - o que cada variável representa;
 - quais valores de teste foram usados;
 - o que se espera que apareça na saída do console.

3) As referências (ABNT)

- Listar os livros e os materiais consultados, com referências **no padrão ABNT**.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	Clareza, correção conceitual e boa explicação das condicionais, dos laços e do case-sensitive.
A estrutura e a organização do código	3,0	Indentação correta, nomes adequados, comentários úteis e organização coerente.
O funcionamento da solução	3,0	Compilação e execução sem erros, com saídas coerentes em todos os testes solicitados.
A criatividade e os aprimoramentos	2,0	Implementação de extensões, validações, melhorias de interação e organização em classes/métodos.

Nota Final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão a nota reduzida proporcionalmente.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): IMPLEMENTANDO UM MINIJOGO DE BATALHA EM CONSOLE COM POO EM C# (CLASSES E OBJETOS)

1) ORIENTAÇÕES GERAIS (ENTREGA)

Cada aluno (ou equipe) deverá entregar **um relatório curto (2 a 3 páginas)** contendo:

1.1) Um resumo teórico (com palavras próprias)

Inclua, obrigatoriamente:

- A definição de **classe** e de **objeto (instância)** e a diferença entre os dois.
- A explicação do que são **propriedades, métodos e construtores** em C#.
- A explicação do que é **encapsulamento**, com um exemplo no seu código (por exemplo, uma propriedade com set restrito).
- A explicação do que é uma **propriedade calculada** (por exemplo, um bool que depende de Vida).
- A descrição de como ocorre a **interação entre objetos** no minijogo (um objeto chamando método de outro e alterando o estado do outro).
- A descrição do uso de **condicionais** e de **laços**, com foco no loop de turnos da batalha.

1.2) O código-fonte completo, com comentários didáticos

- Cole o código final no relatório.
- Faça comentários em trechos relevantes, como:
 - a criação de objetos com new;
 - o construtor e as inicializações;
 - o uso de get, set e private set;
 - o método de ataque e o método de receber dano;
 - o laço while da batalha;
 - as verificações de “estar vivo”.

1.3) Um registro de testes

Inclua pelo menos:

- um teste que mostre **uma batalha completa** contra o primeiro inimigo;
- um teste que mostre **a segunda batalha** (se o jogador sobreviver);
- um teste que confirme que a vida **não fica negativa**.

1.4) As referências em ABNT

- Liste os livros (e, se necessário, páginas oficiais) que você consultou.
- A formatação deve seguir a ABNT (modelo no final deste roteiro).

2) REQUISITOS FUNCIONAIS DO PROGRAMA

Ao final, seu programa deve:

1. Exibir um título do minijogo e uma breve explicação.
2. Criar **um objeto** de jogador (por exemplo, “Herói”) com vida e força iniciais.
3. Criar **dois objetos** de inimigos diferentes (por exemplo, “Slime” e “Goblin”), com vida, força e recompensa.
4. Mostrar o estado inicial do jogador e dos inimigos no console.
5. Executar **duas batalhas em sequência**:
 - o primeira batalha: jogador vs. inimigo 1;
 - o segunda batalha: jogador vs. inimigo 2 (somente se o jogador estiver vivo).
6. Em cada batalha:
 - o ocorrerem turnos;
 - o o jogador atacar;
 - o o inimigo contra-atacar, se ainda estiver vivo;
 - o existir uma pausa por ENTER para avançar os turnos.
7. Ao derrotar um inimigo, o jogador ganhar pontos de recompensa.
8. Exibir o estado final do jogador ao término das batalhas.

3) REQUISITOS DE POO (O QUE DEVE EXISTIR EM CÓDIGO)

Você deve implementar as seguintes classes (com nomes iguais ou equivalentes):

3.1) A classe Jogador

Implemente:

- as propriedades:
 - o Nome (texto);
 - o Vida (inteiro) com **leitura pública e alteração restrita** (por exemplo, private set);
 - o Força (inteiro);
 - o Pontuacao (inteiro) com **alteração restrita**;
 - o EstaVivo (booleano) como **propriedade calculada**, baseada em Vida.
- o construtor:
 - o deve receber nome, vidaInicial e forçaInicial;
 - o deve inicializar Pontuacao com zero.
- os métodos:
 - o Atacar(Inimigo alvo):
 - deve impedir ataque se o jogador estiver sem vida;
 - deve impedir ataque se o alvo já estiver derrotado;
 - deve causar dano igual à força;
 - deve chamar alvo.ReceberDano(...).
 - o ReceberDano(int dano):
 - deve reduzir a vida;
 - deve impedir que a vida fique negativa (mínimo zero);
 - deve escrever o novo valor no console.

- GanharPontos(int pontos):
 - deve somar na pontuação;
 - deve escrever a pontuação no console.

3.2) A classe Inimigo

Implemente:

- as propriedades:
 - Nome;
 - Vida com alteração restrita;
 - Força;
 - PontosDeRecompensa;
 - EstaVivo como propriedade calculada.
- o construtor:
 - deve receber nome, vidaInicial, forçaInicial e pontosDeRecompensa.
- os métodos:
 - Atacar(Jogador alvo) com validações análogas às do jogador;
 - ReceberDano(int dano) com vida mínima zero e mensagens no console.

3.3) A classe Jogo

Implemente:

- o método Executar() que:
 - cria os objetos;
 - imprime os estados iniciais;
 - chama o método de batalha para o primeiro inimigo;
 - encerra se o jogador morrer;
 - chama o método de batalha para o segundo inimigo;
 - imprime o estado final do jogador.
- um método auxiliar Batalhar(Jogador jogador, Inimigo inimigo) que:
 - executa um laço while enquanto ambos estiverem vivos;
 - realiza turnos com ataque e contra-ataque;
 - pausa por ENTER a cada turno;
 - ao final, concede pontos ao jogador se ele vencer.

3.4) A classe Program

Implemente:

- o método Main() que:
 - cria um objeto Jogo;
 - chama Executar().

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	Clareza, correção conceitual e boa explicação das condicionais, dos laços e do case-sensitive.
A estrutura e a organização do código	3,0	Indentação correta, nomes adequados, comentários úteis e organização coerente.
O funcionamento da solução	3,0	Compilação e execução sem erros, com saídas coerentes em todos os testes solicitados.
A criatividade e os aprimoramentos	2,0	Implementação de extensões, validações, melhorias de interação e organização em classes/métodos.

Nota Final: Será a soma ponderada dos itens. Entregas fora do prazo ou com funcionamento mínimo incompleto terão desconto proporcional.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): IMPLEMENTANDO UMA CONTA BANCÁRIA COM POO EM C# (ENCAPSULAMENTO, PROPRIEDADES E MÉTODOS)

1) ORIENTAÇÕES GERAIS (ENTREGA)

Cada aluno (ou equipe) deverá entregar **um relatório curto (2 a 3 páginas)** contendo:

1.1) Um resumo teórico (com palavras próprias)

Inclua, obrigatoriamente:

- A explicação do que é **POO**, com foco em **classe** e **objeto**.
- A explicação do que é **encapsulamento** e por que se evita expor diretamente o estado interno.
- A diferença entre **campo** (por exemplo, saldo) e **propriedade** (por exemplo, Saldo).
- A explicação de um **get público** e de um **set restrito** (por exemplo, private set).
- A justificativa do uso de decimal para valores monetários.
- A descrição do papel de um **construtor** com valor padrão (parâmetro opcional).
- A explicação de como **métodos** podem impor regras de negócio (por exemplo, não aceitar depósito negativo e não permitir saque maior que o saldo).

1.2) O código-fonte completo, com comentários didáticos

- Cole o código final no relatório.
- Comente trechos relevantes:
 - o campo privado;
 - a propriedade de leitura pública;
 - o construtor com saldoInicial = 0m;
 - as validações de depósito e de saque;
 - as alterações no saldo.

1.3) Um registro de testes

Inclua pelo menos:

- um teste de depósito válido;
- um teste de depósito inválido (zero ou negativo);
- um teste de saque válido;
- um teste de saque inválido (maior que o saldo);
- um teste de saque inválido (zero ou negativo).

1.4) As referências em ABNT

- Liste os livros (e, se necessário, páginas oficiais) consultados.
- Formate em ABNT (modelo no final deste roteiro).

2) OBJETIVO DA ATIVIDADE

Construir uma classe ContaBancaria que mantenha um saldo com segurança (encapsulado) e ofereça operações de **depósito** e **saque** com validações, além de uma aplicação Console simples para testar o comportamento.

3) REQUISITOS DO PROGRAMA

3.1) Requisitos de POO (classe principal)

Você deve implementar uma classe chamada ContaBancaria que contenha:

1. **Um campo privado** saldo do tipo decimal.
2. **Uma propriedade pública** Saldo do tipo decimal com:
 - um **get público**;
 - um **set restrito** (por exemplo, private set) ou uma alternativa equivalente que impeça alterações externas.
3. **Um construtor** que receba um saldo inicial do tipo decimal com valor padrão (por exemplo, zero).
4. **Um método** Depositar(decimal quantia) que:
 - rejeite quantias menores ou iguais a zero;
 - some a quantia ao saldo quando válida;
 - retorne true quando efetuar o depósito e false quando rejeitar.
5. **Um método** Sacar(decimal quantia) que:
 - rejeite quantias menores ou iguais a zero;
 - rejeite saques acima do saldo;
 - subtraia a quantia do saldo quando válida;
 - retorne true quando efetuar o saque e false quando rejeitar.

3.2) Requisitos de aplicação (Console)

No Main, crie um pequeno roteiro de execução que:

- crie pelo menos **uma** conta com saldo inicial;
- tente realizar depósitos e saques válidos e inválidos;
- imprima no console:
 - o saldo inicial;
 - o resultado (sucesso/fracasso) de cada operação;
 - o saldo após cada operação.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	A clareza e a correção dos conceitos de classe, objeto, encapsulamento, campo, propriedade, construtor e métodos.
A estrutura e a organização do código	3,0	A indentação correta, os nomes adequados, os comentários úteis e a separação clara entre a classe e o programa de testes.
O funcionamento da solução	3,0	A execução sem erros, as validações corretas, o saldo consistente e os testes demonstrando os casos válidos e inválidos.
A criatividade e os aprimoramentos	2,0	A implementação de extensões opcionais com consistência e boa integração ao código existente.

Nota Final: Será a soma ponderada dos itens. Entregas fora do prazo ou com funcionamento mínimo incompleto terão desconto proporcional.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ABSTRAÇÃO, HERANÇA E POLIMORFISMO EM C# (APLICAÇÃO DE CONSOLE)

OBJETIVO

Implementar, em uma aplicação de console, uma hierarquia simples de classes para modelar animais, usando:

- a classe abstrata (para definir um “contrato” comum);
- a herança (para reutilizar e especializar comportamento);
- o polimorfismo (para chamar o mesmo método em objetos de tipos diferentes).

ORIENTAÇÕES

Cada aluno (ou equipe) deve produzir **um relatório curto (1 a 2 páginas)** contendo:

1) O resumo teórico

No texto, explique com palavras próprias:

- o que é **Programação Orientada a Objetos (POO)** e o que se pretende ao usar POO;
- o que é **uma classe abstrata** e por que ela é útil em um projeto;
- a diferença entre **método abstrato** e **método virtual** (se aparecer na sua explicação, compare corretamente);
- o que é **herança** e quando ela faz sentido;
- o que é **polimorfismo** no contexto de “uma variável do tipo base apontando para um objeto do tipo derivado”;
- o papel de **construtores** em uma classe base e em uma classe derivada;
- o que é **override** e por que ele é exigido ao implementar um método abstrato.

2) O código-fonte comentado

- inserir **o código-fonte completo** da solução produzida;
- comentar as linhas mais relevantes, com foco em:
 - a declaração da classe abstrata;
 - a propriedade que guarda o nome;
 - o construtor da classe base e a chamada do base(...) no construtor das subclasses;
 - a sobrescrita (override) do método que emite som;
 - os pontos do programa em que aparece o polimorfismo.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	A clareza, a correção dos conceitos e a conexão entre abstração, herança e polimorfismo.
A estrutura e a organização do código	3,0	A identificação correta, os nomes adequados, a separação coerente em classes e os comentários necessários.
O funcionamento da solução	3,0	A compilação sem erros e a execução com saídas coerentes, incluindo chamadas polimórficas.
A criatividade e os aprimoramentos	2,0	A inclusão de extensões (por exemplo, novos animais, listas, entradas do usuário, validações) com qualidade.

Nota Final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não entregarem no prazo ou não atingirem o funcionamento mínimo terão desconto proporcional na nota.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): INTERFACES E POLIMORFISMO EM C# (GEOMETRIA EM APLICAÇÃO DE CONSOLE)

OBJETIVO

Construir uma aplicação de console que modele **formas geométricas** por meio de **uma interface**, com classes concretas que a implementam, e que use **polimorfismo** ao armazenar objetos diferentes em uma coleção do tipo List<IForma>.

ORIENTAÇÕES

Cada aluno (ou equipe) deve produzir **um relatório curto (1 a 2 páginas)** contendo:

1) O resumo teórico

Explique, com palavras próprias:

- o que é **uma interface** em POO e qual é a ideia de “contrato” (quais métodos devem existir);
- a diferença entre **interface** e **classe abstrata** (com exemplos do que cada uma costuma resolver);
- o que é **implementação** de interface em uma classe concreta;
- o que é **polimorfismo** com interface (por exemplo, “uma variável do tipo IForma pode referir-se a objetos de classes diferentes”);
- o papel de **propriedades** e de **construtores** para guardar e inicializar os dados (largura, altura, raio);
- a diferença entre **área** e **perímetro**, com as fórmulas que você usou no código.

2) O código-fonte comentado

- inserir **o código-fonte completo** da solução produzida;
- comentar as partes principais, incluindo:
 - a interface e as assinaturas dos métodos;
 - as propriedades e os construtores das classes;
 - os cálculos em CalcularArea() e em CalcularPerimetro();
 - o uso de List<IForma> e do laço que percorre as formas;
 - a impressão formatada no console.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	A correção conceitual sobre interfaces, implementação e polimorfismo, com explicações claras.
A estrutura e a organização do código	3,0	A boa organização em interface e classes, a identificação correta, os nomes adequados e os comentários necessários.
O funcionamento da solução	3,0	A compilação sem erros, a execução correta e os cálculos coerentes de área e de perímetro.
A criatividade e os aprimoramentos	2,0	A inclusão de extensões com validações, novas formas, entradas do usuário ou melhor apresentação da saída.

Nota Final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não entregarem no prazo ou não atingirem o funcionamento mínimo terão desconto proporcional na nota.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): Modelando herança e polimorfismo com as classes Animal e Cachorro em C#

Objetivo geral

Implementar, compilar e executar um pequeno exemplo de Programação Orientada a Objetos em C#, usando **uma classe base** e **uma classe derivada**, para praticar **as propriedades, os métodos, a herança e o polimorfismo** (com virtual e override).

Orientações para a entrega (relatório curto, de 1 a 2 páginas)

Cada aluno (ou equipe) deve entregar um relatório contendo:

1) Um resumo teórico (com palavras próprias)

Inclua, de forma objetiva:

- **A definição de classe e de objeto**, distinguindo “o modelo” (a classe) de “uma instância” (o objeto).
- **A explicação de encapsulamento**, destacando o papel de **propriedades** (por exemplo, Nome) e de **métodos** (por exemplo, Comer()).
- **A explicação de herança**, descrevendo como uma classe derivada reaproveita e especializa uma classe base.
- **A explicação de polimorfismo**, relacionando:
 - **o método virtual na classe base**, e
 - **o método override na classe derivada**, e descrevendo o que muda no comportamento quando o método é sobrescrito.

2) Um código-fonte comentado

- Inserir **o código-fonte completo** que você produziu.
- Comentar **as linhas principais**, incluindo:
 - a declaração da **classe base**,
 - a declaração da **classe derivada**,
 - a propriedade Nome,
 - a chamada de Console.WriteLine(...),
 - o uso de virtual e de override,
 - a criação de objetos (instâncias) e a chamada de métodos no Main.

3) As referências em ABNT

- Listar **os livros e os materiais consultados**.
- Formatar as referências conforme **as regras da ABNT**.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	A clareza, a correção conceitual e a relação entre classe, objeto, herança e polimorfismo.
A estrutura e a organização do código	3,0	A organização, os nomes adequados, os comentários úteis e a legibilidade.
O funcionamento da solução	3,0	A execução sem erros, a criação correta das classes e o uso correto de virtual e override.
A criatividade e os aprimoramentos	2,0	A inclusão de extensões, como um novo animal, uma lista polimórfica e validações.

Nota Final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão sua nota diminuída proporcionalmente.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): Herança, polimorfismo e casting com Animal e Cachorro em C#

Orientações

Cada aluno (ou equipe) deve produzir um relatório curto (1 a 2 páginas) contendo:

1) Um resumo teórico (com palavras próprias)

No resumo, inclua:

- **A definição de classe e de objeto**, explicando a relação entre “tipo” e “instância”.
- **A explicação de herança**, descrevendo o que significa Cachorro : Animal e o que é reutilizado.
- **A explicação de polimorfismo**, descrevendo:
 - o que significa marcar um método como virtual na classe base;
 - o que significa marcar um método como override na classe derivada;
 - por que a chamada de FazerSom() pode executar implementações diferentes, dependendo do tipo real do objeto.
- **A explicação de referência estática e tipo dinâmico**, usando um exemplo do tipo:
 - “uma variável do tipo Animal pode referenciar um objeto do tipo Cachorro”.
- **A explicação de casting (conversão de tipo)**, respondendo:
 - por que animal2.AbanarRabo() não compila;
 - por que ((Cachorro)animal2).AbanarRabo() compila;
 - o que acontece se o objeto não for realmente um Cachorro (erro em tempo de execução).
- **A distinção entre erro de compilação e erro em tempo de execução**, aplicando aos trechos do programa.

2) Um código-fonte comentado

- Inserir o código-fonte completo que você produziu.
- Comentar as partes essenciais:
 - as classes Animal e Cachorro;
 - a propriedade Nome;
 - o método Comer() e as mensagens no console;
 - o método virtual FazerSom() e o override em Cachorro;
 - a criação de animal1 e de animal2;
 - a diferença entre Animal animal2 = new Cachorro(); e Animal animal1 = new Animal();;
 - o motivo do comentário que indica erro em animal2.AbanarRabo();
 - o casting ((Cachorro)animal2) e a chamada de AbanarRabo() após o cast.

3) As referências em ABNT

- Listar os livros e os materiais consultados, em formato ABNT.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
A qualidade do resumo teórico	2,0	A correção e a clareza nos conceitos de classe, objeto, herança, polimorfismo, casting e tipos.
A estrutura e a organização do código	3,0	A indentação, os nomes claros, os comentários úteis e a organização em blocos.
O funcionamento da solução	3,0	A compilação e a execução sem erros, com saídas corretas e demonstrações pedidas (inclusive o cast).
A criatividade e os aprimoramentos	2,0	A inclusão de melhorias, como verificação de tipo com is, uso de as, novas classes e listas polimórficas.

Nota Final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão sua nota diminuída proporcionalmente.